

ARQUITETURA DOS TRANSFORMADORES

Editado por: Dr. Arnaldo de Carvalho Junior

Data: Setembro 23, 2025.

A arquitetura dos transformadores (*Transformers*) explicada.

Em aprendizado de máquinas (*Machine Learning* - ML), redes Transformers são um tipo de arquitetura de rede neural projetada principalmente para lidar com sequências de dados, como texto, áudio ou séries temporais, de forma paralela e eficiente. Elas revolucionaram a área de Processamento de Linguagem Natural (NLP) e se tornaram a base de modelos como GPT, BERT, T5 e muitos outros. O básico da arquitetura do transformador reside no conceito de codificador/decodificador.

1. O que é o Transformador?

Um transformador é uma rede neural que se destaca em entender o contexto de dados sequenciais e gerar novos dados a partir dela. Foi primeiro introduzido por Vaswani et al [1] em 2017.

Eles são os primeiros a confiar apenas em auto-distribuição, sem usar redes neurais recorrentes (*recurrent neural networks* - RNNs) ou redes neurais convolucionais (*convolution neural networks* – CNNs).

2. Transformador como uma Caixa Preta

Pode-se imaginar um transformador para a tradução de idiomas como uma caixa-preta.

- Entrada: uma frase em um idioma.
- Saída: sua tradução.

Mas o que acontece dentro desta caixa-preta?

3. Arquitetura Codificador / Decodificador (*Encoder / Decoder Architecture*)

- Entrada: frase em espanhol “De quién es?”
- Codificador: Recebe a sequência de entrada (por exemplo, uma frase). Processa e gera uma representação vetorial contextualizada de cada elemento da sequência. É composto por camadas de atenção e feed-forward. O transforma em um formato estruturado que captura sua essência.

- Decodificador: Utilizado em tarefas de tradução ou geração de texto. Recebe a representação do *encoder* e produz a sequência de saída, passo a passo. Por exemplo, recebe os dados codificados do exemplo e gera a tradução.
- Saída: a frase traduzida: “De quem é?”

4. A Arquitetura Por Trás dos Transformadores

Cada codificador e decodificador são compostos de camadas. A Figura 1 apresenta a arquitetura do Transformer. Aqui está como eles funcionam:

- Codificadores: processe a entrada sequencialmente, camada por camada.
- Decodificadores: pegue os dados codificados e gere o passo a passo.

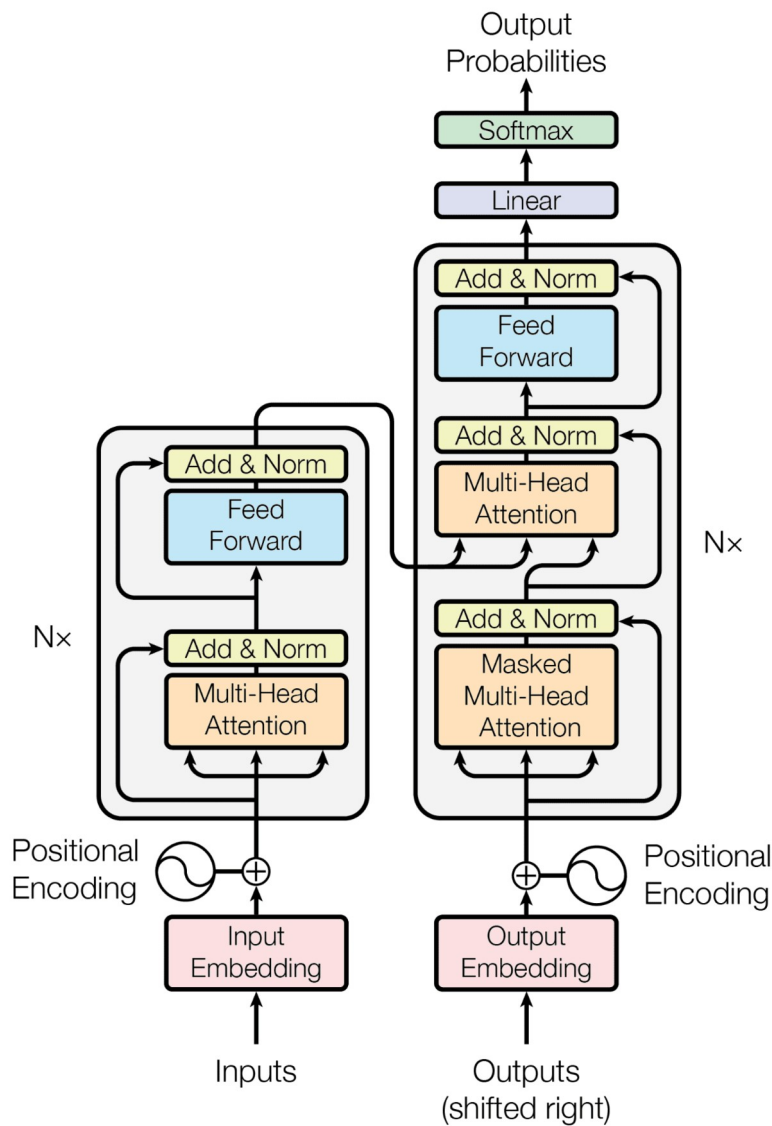


Figura 1 – O Modelo Transformer
Adaptado de [2].

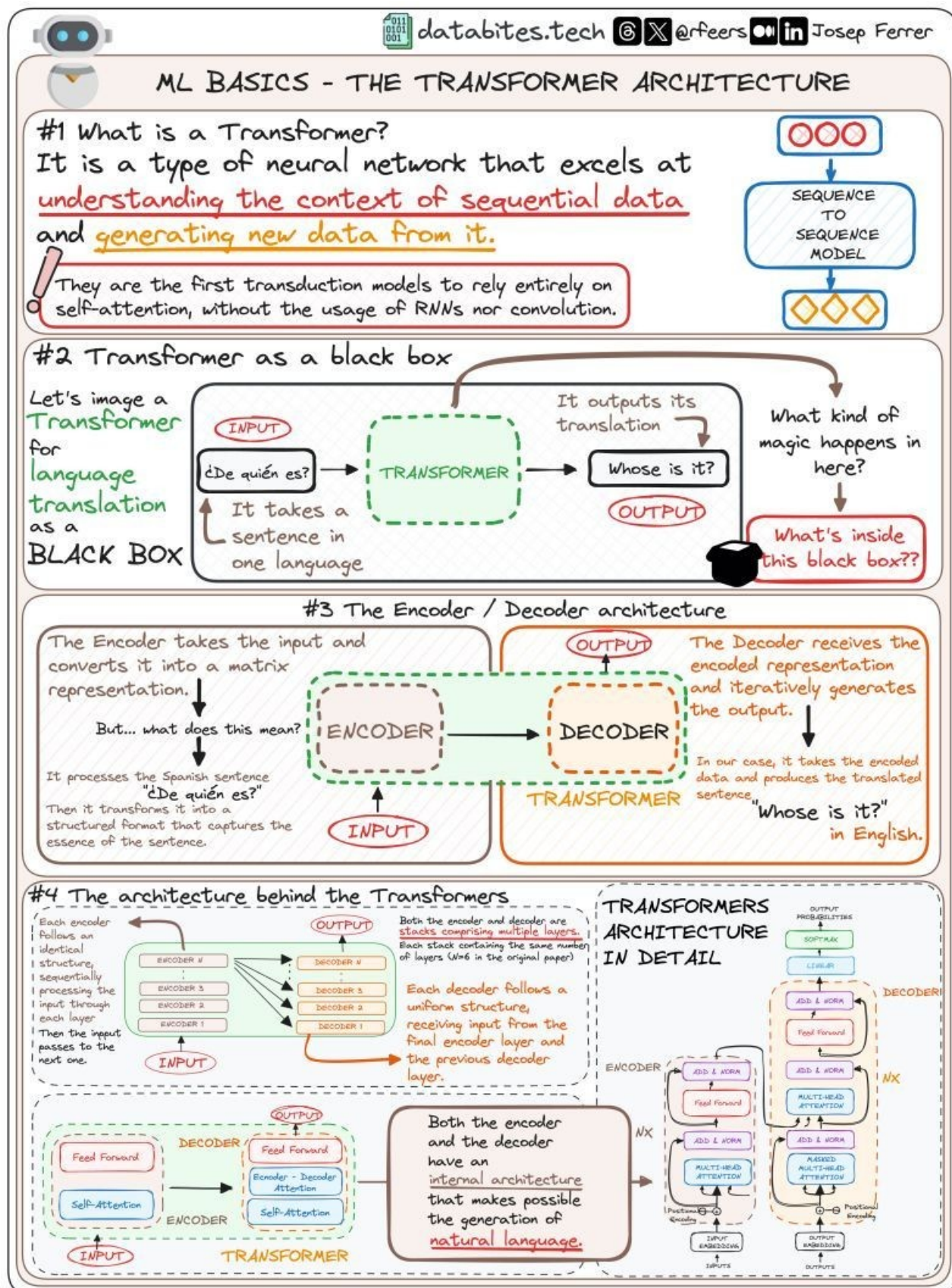


Figura 2 – Arquitetura Transformer Explicada.
Adaptado de [3].

5. Vantagens sobre RNNs e LSTMs

Antes dos Transformers, redes recorrentes (RNNs) e LSTMs eram populares para sequências, mas tinham limitações:

- Processamento sequencial lento.
- Difícil capturar dependências de longo alcance em sequências longas.

Os Transformers:

- Processam toda a sequência simultaneamente (paralelização).
- Capturam relações complexas de longo alcance de forma eficiente.
- Escalam muito melhor com dados grandes.

Ambos usam redes neurais de autoatendimento e avanço, permitindo a geração de linguagem natural.

A Figura 2 apresenta uma representação gráfica explicada dos blocos que compõem o codificador e decodificador do Transformer [3].

O Anexo apresenta um exemplo de rede transformer para análise de sequência de dados fictícios. O programa utiliza biblioteca PyTorch e executa 100 épocas [4], [5].

REFERÊNCIAS:

[1] VASWANI, Ashish et al. Attention is all you need. Advances in neural information processing systems, v. 30, n. 1, p. 5998-6008, 2017. Disponível em: <<https://arxiv.org/abs/1706.03762>>. Acessado em Set 23, 2025.

[2] STEFANIA, C. The Transformer Model. Machine Learning mastery. Disponível em: <https://machinelearningmastery.com/the-transformer-model/>. Acessado em Jul 30, 2024.

[3] FERRER, J The Transformers Architecture Clearly Explained. Machine learning Community (Linkedin Group), Jul 2024. Disponível em: https://media.licdn.com/dms/image/D4D22AQEP0sRasr5ijg/feedshare-shrink_2048_1536/0/1722175071467?e=1725494400&v=beta&t=I16mvO8m79ITackZwwNlxsiblfEcM_koaRLuQeqQ1Ds. Acessado em Jul 30, 2024.

[4] GEEKSFORGEES. Transformer using PyTorch, 2025. Disponível em: <<https://www.geeksforgeeks.org/deep-learning/transformer-using-pytorch/>>. Acessado em Set 23, 2025.

[5] SARKAR, Arjun, Transformer Model Tutorial in PyTorch: From Theory to Code, Datacamp, 2025. Disponível em: <<https://www.datacamp.com/tutorial/building-a-transformer-with-py-torch>>. Acessado em Set 23, 2025.

APÊNDICE A – Exemplo de Programa de Rede Transformer

```
# Referência: Transformer using PyTorch

# Disponível em: https://www.geeksforgeeks.org/deep-learning/transformer-using-pytorch/
# Referência: Transformer Model Tutorial in PyTorch: From Theory to Code
# Disponível em: https://www.datacamp.com/tutorial/building-a-transformer-with-pytorch
# Adaptado por: Dr. Arnaldo de Carvalho Junior - 23/Set/2025

# Construindo a Arquitetura Transformer usando PyTorch
# Importando as bibliotecas
import torch
from torch import nn as nn
from torch import optim as optim
import torch.utils.data as data
import math
import copy

# Bloco de classe Multihead Attention
# Mecanismo para focar em diferentes partes da entrada (input)
# Captura dependências através de diferentes posições na sequência
# Ao fazer isso, o modelo pode capturar vários relacionamentos nos dados de
# entrada em diferentes escalas, melhorando a capacidade expressiva do modelo
class MultiHeadAttention(nn.Module):
    def __init__(self, d_model, num_heads):
        super(MultiHeadAttention, self).__init__()
        assert d_model % num_heads == 0, "d_model must be divisible by num_heads"
        self.d_model = d_model
        self.num_heads = num_heads
        self.d_k = d_model // num_heads
        self.W_q = nn.Linear(d_model, d_model)
        self.W_k = nn.Linear(d_model, d_model)
        self.W_v = nn.Linear(d_model, d_model)
        self.W_o = nn.Linear(d_model, d_model)

    def scaled_dot_product_attention(self, Q, K, V, mask=None):
        attn_scores = torch.matmul(Q, K.transpose(-2, -1)) / math.sqrt(self.d_k)
        if mask is not None:
            attn_scores = attn_scores.masked_fill(mask == 0, -1e9)
        attn_probs = torch.softmax(attn_scores, dim=-1)
        output = torch.matmul(attn_probs, V)
        return output

    def split_heads(self, x):
        batch_size, seq_length, d_model = x.size()
```



```

        return x.view(batch_size, seq_length, self.num_heads, self.d_k).transpose(1, 2)

def combine_heads(self, x):
    batch_size, _, seq_length, d_k = x.size()
    return x.transpose(1, 2).contiguous().view(batch_size, seq_length, self.d_model)

def forward(self, Q, K, V, mask=None):
    Q = self.split_heads(self.W_q(Q))
    K = self.split_heads(self.W_k(K))
    V = self.split_heads(self.W_v(V))
    attn_output = self.scaled_dot_product_attention(Q, K, V, mask)
    output = self.W_o(self.combine_heads(attn_output))
    return output

# Bloco de rede Position-Wise Feed-Forward com ativação ReLU
# Camadas totalmente conectadas em termos de posição
# define uma rede neural de avanço em posição de posição que consiste em duas
# camadas lineares com uma função de ativação do ReLU no meio
# Transforma as saídas de atenção, adicionando complexidade
# Ajuda a transformar os recursos aprendidos pelos mecanismos de atenção no
# transformador, atuando como uma etapa adicional de processamento para as
# saídas de atenção
class PositionWiseFeedForward(nn.Module):
    def __init__(self, d_model, d_ff):
        super(PositionWiseFeedForward, self).__init__()
        self.fc1 = nn.Linear(d_model, d_ff)
        self.fc2 = nn.Linear(d_ff, d_model)
        self.relu = nn.ReLU()

    def forward(self, x):
        return self.fc2(self.relu(self.fc1(x)))

# Bloco de codificação posicional (positional encoding)
# Adiciona informações posicionais às incorporações
# Fornece contexto de ordem de sequência para o modelo
# Usa funções senoidais e cosseno de diferentes frequências para gerar a
# codificação posicional
class PositionalEncoding(nn.Module):
    def __init__(self, d_model, max_seq_length):
        super(PositionalEncoding, self).__init__()
        pe = torch.zeros(max_seq_length, d_model)
        position = torch.arange(0, max_seq_length, dtype=torch.float).unsqueeze(1)
        div_term = torch.exp(torch.arange(0, d_model, 2).float() * -(math.log(10000.0) /
d_model))
        pe[:, 0::2] = torch.sin(position * div_term) # seno
        pe[:, 1::2] = torch.cos(position * div_term) # cosseno
        self.register_buffer('pe', pe.unsqueeze(0))

```

```

def forward(self, x):
    return x + self.pe[:, :x.size(1)]

# Camada de Codificação (encoder)
# Normalização: normaliza entradas para cada sub-camada. Estabiliza treinamento,
# melhora a convergência
# Residual Connections: Os atalhos de conexões residuais entre camadas ajudam a
# treinar redes mais profundas, minimizando problemas de gradiente
# Dropout: Zeros aleatoriamente, algumas conexões de rede evitam o excesso de
# ajuste, regularizando o modelo
class EncoderLayer(nn.Module):
    # Inicialização
    def __init__(self, d_model, num_heads, d_ff, dropout):
        super(EncoderLayer, self).__init__()
        self.self_attn = MultiHeadAttention(d_model, num_heads)
        self.feed_forward = PositionWiseFeedForward(d_model, d_ff)
        self.norm1 = nn.LayerNorm(d_model)
        self.norm2 = nn.LayerNorm(d_model)
        self.dropout = nn.Dropout(dropout)

    # Método Forward
    def forward(self, x, mask):
        attn_output = self.self_attn(x, x, x, mask)
        x = self.norm1(x + self.dropout(attn_output))
        ff_output = self.feed_forward(x)
        x = self.norm2(x + self.dropout(ff_output))
        return x

# Camada de Decodificação (decoder)
class DecoderLayer(nn.Module):
    # Inicialização
    def __init__(self, d_model, num_heads, d_ff, dropout):
        super(DecoderLayer, self).__init__()
        self.self_attn = MultiHeadAttention(d_model, num_heads)
        self.cross_attn = MultiHeadAttention(d_model, num_heads)
        self.feed_forward = PositionWiseFeedForward(d_model, d_ff)
        self.norm1 = nn.LayerNorm(d_model)
        self.norm2 = nn.LayerNorm(d_model)
        self.norm3 = nn.LayerNorm(d_model)
        self.dropout = nn.Dropout(dropout)

    # Método Forward
    def forward(self, x, enc_output, src_mask, tgt_mask):
        attn_output = self.self_attn(x, x, x, tgt_mask)
        x = self.norm1(x + self.dropout(attn_output))
        attn_output = self.cross_attn(x, enc_output, enc_output, src_mask)
        x = self.norm2(x + self.dropout(attn_output))
        ff_output = self.feed_forward(x)
        x = self.norm3(x + self.dropout(ff_output))

```

```

    return x

# Modelo Transformador (Transformer)
class Transformer(nn.Module):
    # Inicialização
    def __init__(self, src_vocab_size, tgt_vocab_size, d_model, num_heads,
num_layers, d_ff, max_seq_length, dropout):
        super(Transformer, self).__init__()
        self.encoder_embedding = nn.Embedding(src_vocab_size, d_model)
        self.decoder_embedding = nn.Embedding(tgt_vocab_size, d_model)
        self.positional_encoding = PositionalEncoding(d_model, max_seq_length)

        self.encoder_layers = nn.ModuleList([EncoderLayer(d_model, num_heads, d_ff,
dropout) for _ in range(num_layers)])
        self.decoder_layers = nn.ModuleList([DecoderLayer(d_model, num_heads, d_ff,
dropout) for _ in range(num_layers)])

        self.fc = nn.Linear(d_model, tgt_vocab_size)
        self.dropout = nn.Dropout(dropout)

    # Gerando o método de máscara
    def generate_mask(self, src, tgt):
        src_mask = (src != 0).unsqueeze(1).unsqueeze(2)
        tgt_mask = (tgt != 0).unsqueeze(1).unsqueeze(3)
        seq_length = tgt.size(1)
        nopeak_mask = (1 - torch.triu(torch.ones(1, seq_length, seq_length),
diagonal=1)).bool()
        tgt_mask = tgt_mask & nopeak_mask
        return src_mask, tgt_mask

    # Método Forward
    def forward(self, src, tgt):
        src_mask, tgt_mask = self.generate_mask(src, tgt)
        src_embedded =
self.dropout(self.positional_encoding(self.encoder_embedding(src)))
        tgt_embedded =
self.dropout(self.positional_encoding(self.decoder_embedding(tgt)))

        enc_output = src_embedded
        for enc_layer in self.encoder_layers:
            enc_output = enc_layer(enc_output, src_mask)

        dec_output = tgt_embedded
        for dec_layer in self.decoder_layers:
            dec_output = dec_layer(dec_output, enc_output, src_mask, tgt_mask)

    # Saída: A saída final é um tensor que representa as previsões do modelo para
    # a sequência de destino.

```



```

        output = self.fc(dec_output)
        return output

# Exemplo
# Amostra de preparação de dados
# Para fins ilustrativos, um conjunto de dados fictícios será criado
src_vocab_size = 1000
tgt_vocab_size = 1000
d_model = 512
num_heads = 8
num_layers = 6
d_ff = 2048
max_seq_length = 100
dropout = 0.1

# Criando uma instância transformadora (transformer)
transformer = Transformer(src_vocab_size, tgt_vocab_size, d_model, num_heads,
                           num_layers, d_ff, max_seq_length, dropout)

# Gerando alguns dados fictícios de amostra aleatórios
batch_size = 64
src_data = torch.randint(1, src_vocab_size, (batch_size, max_seq_length)) #
(batch_size, seq_length)
tgt_data = torch.randint(1, tgt_vocab_size, (batch_size, max_seq_length)) #
(batch_size, seq_length)

# Treinando o Modelo
# Função perda (loss) e otimizador
# Nesse exemplo a função perda é cross-entropy loss
criterion = nn.CrossEntropyLoss(ignore_index=0)
# Define o otimizador como Adam com uma taxa de aprendizado (lr) de 0,0001 e
# valores beta específicos.
optimizer = optim.Adam(transformer.parameters(), lr=0.0001, betas=(0.9, 0.98),
                        eps=1e-9)

# Treinamento para 100 épocas
transformer.train()
for epoch in range(100):
    optimizer.zero_grad()
    output = transformer(src_data, tgt_data[:, :-1])
    loss = criterion(output.contiguous().view(-1, tgt_vocab_size), tgt_data[:,
1:].contiguous().view(-1))
    loss.backward()
    optimizer.step()
    print(f"Epoch: {epoch+1}, Loss: {loss.item()}")

# Avaliando o Modelo
transformer.eval()

```

```

# # Gerando dados de validação de amostra aleatória
val_src_data = torch.randint(1, src_vocab_size, (64, max_seq_length)) #
(batch_size, seq_length)
val_tgt_data = torch.randint(1, tgt_vocab_size, (64, max_seq_length)) #
(batch_size, seq_length)

with torch.no_grad():
    val_output = transformer(val_src_data, val_tgt_data[:, :-1])
    val_loss = criterion(val_output.contiguous().view(-1, tgt_vocab_size),
val_tgt_data[:, 1:].contiguous().view(-1))
    print(f"Validation Loss: {val_loss.item()}")

```